



**UNIVERSIDADE DE PERNAMBUCO
LICENCIATURA EM COMPUTAÇÃO**

**ANDRISSON HEITOR SOUSA DE MELO
ISAQUE ALVES PEREIRA
VITOR MANOEL FERREIRA LINO
RAFAEL GUEIROS SABINO
PAULO RICARDO DA SILVA FERREIRA**

PROJETO DE PROGRAMAÇÃO 1

GARANHUNS 2026

02

SUMÁRIO

1. INTRODUÇÃO	03
2. FERRAMENTAS	03
3. MINI MUNDO	03
4. ARQUITETURA DO PROJETO	04
4.1. ESTRUTURA DE PASTAS	04
4.2. FLUXO GERAL DO SISTEMA	04
5. DOCUMENTAÇÃO DOS MÓDULOS	04
5.1. CRUD DE USUÁRIOS	04
5.2. CRUD DE HÁBITOS	06
5.3. CRUD DE REGISTROS DIÁRIOS	07
5.4. MÓDULOS DE RELATÓRIOS	09
6. PERSISTÊNCIA DE DADOS	11
7. VALIDAÇÃO E TRATAMENTO DE ERROS	12
8. CONSIDERAÇÕES FINAIS	12

Habit Flow

Sistema de Hábitos Saudáveis

Trabalho apresentado à Universidade de Pernambuco (UPE) como requisito para conclusão da disciplina de Programação 1 do curso de Licenciatura em Computação.

1. Introdução

A construção de hábitos saudáveis é um desafio recorrente no dia a dia das pessoas, e ferramentas de acompanhamento podem ajudar significativamente nesse processo. Este trabalho tem como objetivo desenvolver um sistema de linha de comando (CLI) em Python, chamado Habit Flow, capaz de gerenciar usuários, hábitos e registros diários, permitindo o acompanhamento da evolução de cada usuário ao longo do tempo.

O sistema foi desenvolvido em grupo, com cada integrante responsável por um módulo específico do projeto, seguindo uma arquitetura modular que facilita a manutenção e a expansão futura do código.

2. Ferramentas

Para o desenvolvimento do projeto, foi utilizada a linguagem **Python 3**, com persistência de dados em arquivos **JSON** (sem uso de banco de dados relacional). O ambiente de desenvolvimento utilizado foi o **VS Code**, com controle de versão feito via **Git** e hospedagem do repositório no **GitHub**, utilizando branches individuais por módulo e Pull Requests para integração do código.

3. Mini Mundo

Um grupo de estudantes precisa de uma ferramenta simples para acompanhar hábitos saudáveis no dia a dia (como beber água, praticar exercícios, dormir bem, etc.). O sistema deve permitir o cadastro de usuários, o cadastro dos hábitos que cada um deseja acompanhar, o registro diário do cumprimento (ou não) desses hábitos, e a geração de relatórios que mostrem a evolução e a consistência de cada usuário ao longo do tempo. Para isso, o sistema foi dividido em quatro módulos principais: gerenciamento de usuários, gerenciamento de hábitos, registro diário de cumprimento dos hábitos, e geração de relatórios — cada um desenvolvido por um integrante do grupo, de forma independente, mas integrada ao final.

4. Arquitetura do Projeto no GITHUB

4.1. Estrutura de Pastas

projeto_sistema_habitos_saudaveis/

- |— README.md
- |— .gitignore
- |— Projeto completo/
- |— arquivos.py
- |— habitos_isaque.py
- |— usuarios_rafael.py
- |— registros_vitor.py
- |— relatorios_paulo.py
- |— menu_exibicao.py
- |— main_rodar_projeto.py
- |— modulos/
- |— crud_habitos/
 - |— habitos.py
- |— crud_registros/
 - |— registros.py
- |— crud_usuarios/
 - |— usuarios_rafael.py
 - |— arquivos.py
- |— relatorios/
- |— relatorios.py

4.2. Fluxo Geral do Sistema

Cada módulo possui suas próprias funções de cadastro, listagem, busca, atualização e exclusão (CRUD), seguindo o mesmo padrão de validação de entrada e persistência em arquivo JSON.

5. Documentação dos Módulos

5.1. CRUD de Usuários

****Responsável:**** Rafael Gueiros Sabino

Módulo responsável pelo cadastro e gerenciamento dos usuários do sistema.

Funções Principais:

O módulo de usuários é responsável pelas operações básicas de cadastro e gerenciamento. As principais funções implementadas são:

- **cadastrar_usuario()**: realiza o cadastro de um novo usuário, gerando automaticamente o ID com base no maior valor existente (`max(id) + 1`);
- **listar_usuarios()**: exibe todos os usuários cadastrados;
- **buscar_usuario()**: permite localizar um usuário específico a partir do seu ID;
- **atualizar_usuario()**: possibilita a edição dos dados de um usuário já existente;
- **excluir_usuario()**: remove um usuário com base no ID, utilizando filtragem segura da lista.

Dados Armazenados

Cada usuário possui os seguintes atributos:

- `id`
- `nome`
- `idade`
- `peso`
- `altura`

Validação de Dados

Para garantir a integridade das informações inseridas, foram implementadas funções auxiliares:

- **ler_nome()**: impede a inserção de nomes vazios;
- **ler_inteiro_positivo()**: valida entradas numéricas inteiras, tratando erros com `ValueError`;
- **ler_float_positivo()**: valida entradas numéricas decimais, também com tratamento de exceções.

Persistência

Os dados são armazenados no arquivo `usuarios.json`, sendo manipulados pelas funções `carregar_dados()` e `salvar_dados()`, definidas no módulo `arquivos.py`

5.2. CRUD de Hábitos

****Responsável:**** Isaque Alves Pereira

Módulo Responsável pelo cadastro e gerenciamento de hábitos dos usuários no sistema.

O módulo de hábitos é responsável pelo cadastro, consulta, atualização e exclusão dos hábitos saudáveis registrados pelos usuários do sistema Habit Flow. Os dados são armazenados no arquivo **habitos.json** e cada hábito é associado a um usuário previamente cadastrado no sistema.

Funcionalidades

Criar Hábito (criar_habito**)**

Permite cadastrar um novo hábito para um usuário existente. Durante o cadastro, o sistema valida o ID do usuário, solicita o nome do hábito, sua categoria e a meta diária desejada. Cada hábito recebe um identificador único (ID) gerado automaticamente.

Dados registrados:

- ID do hábito;
- ID do usuário;
- Nome do hábito;
- Categoria;
- Meta diária.

Listar Todos os Hábitos (ler_todos_habitos**)**

Exibe todos os hábitos cadastrados no sistema, apresentando suas informações completas, incluindo identificação do usuário, categoria e meta diária.

Consultar Hábito por ID (ler_um_habito**)**

Permite buscar um hábito específico através do seu identificador (ID). Caso o hábito exista, seus detalhes são exibidos ao usuário.

Editar Hábito (editar_habito**)**

Permite atualizar as informações de um hábito já cadastrado. O usuário pode alterar o nome, a categoria e a meta diária, mantendo os valores atuais caso não deseje modificá-los.

Excluir Hábito (**excluir_habito**)

Remove permanentemente um hábito do sistema por meio do seu ID. Após a confirmação da exclusão, os dados são atualizados no arquivo de armazenamento.

Validações Implementadas

- Verificação da existência do usuário antes do cadastro do hábito;
- Validação de IDs numéricos e positivos;
- Impedimento de campos obrigatórios vazios;
- Restrição das categorias às opções permitidas:
 - Alimentação;
 - Sono;
 - Atividade Física;
 - Outro.

Estrutura dos Dados

Cada hábito é armazenado no formato JSON conforme o exemplo abaixo:

```
{  
  "id": 1,  
  "id_usuario": 1,  
  "nome": "Beber Água",  
  "categoria": "alimentação",  
  "meta_diaria": "8 copos"  
}
```

Este módulo tem como objetivo auxiliar os usuários no acompanhamento e na organização de hábitos saudáveis, permitindo o monitoramento contínuo de metas relacionadas à saúde e bem-estar.

5.3. CRUD de Registros Diários

****Responsável:** Vitor Manoel**

Módulo Responsável pelo cadastro e gerenciamento de registros diários no sistema.

O módulo de registros é responsável pelo controle do acompanhamento diário dos hábitos cadastrados no sistema Habit Flow. Ele permite registrar se um hábito foi realizado em determinada data, além de possibilitar a consulta, atualização e exclusão desses registros.

Os dados são armazenados no arquivo `registros.json` e estão diretamente relacionados aos usuários e hábitos previamente cadastrados.

Funcionalidades

Criar Registro (`criar_registro`)

Permite registrar a execução de um hábito em uma data específica. O sistema valida a existência do usuário e do hábito antes de criar o registro, garantindo a integridade dos dados.

Durante o cadastro, o usuário informa:

- ID do usuário;
- ID do hábito;
- Data do registro;
- Se o hábito foi realizado (Sim/Não).

Cada registro recebe um ID único gerado automaticamente.

Listar Registros (`listar_registros`)

Exibe todos os registros cadastrados no sistema, mostrando informações como usuário, hábito, data e status de realização.

Buscar Registro por ID (`procurar_registro`)

Permite buscar um registro específico através do seu ID. Caso encontrado, exibe todas as informações relacionadas ao registro.

Atualizar Registro (`atualizar_registro`)

Permite editar um registro existente. O usuário pode alterar:

- Data do registro;
- Status de realização do hábito (Sim/Não).

O sistema valida novamente os dados inseridos antes de salvar as alterações.

Excluir Registro (`excluir_registro`)

Remove permanentemente um registro do sistema com base no seu ID. Após a exclusão, os dados são atualizados no arquivo JSON.

Validações Implementadas

- Verificação de existência de usuário e hábito antes do registro;
- Validação de IDs numéricos e positivos;
- Validação de data no formato **dd/mm/aaaa**;
- Restrição do campo "realizado" para valores válidos (Sim ou Não);
- Tratamento de entradas inválidas.

Estrutura dos Dados

Cada registro é armazenado no formato JSON conforme o exemplo abaixo:

```
{
  "id": 1,
  "id_usuario": 1,
  "id_habito": 2,
  "data": "24/06/2026",
  "realizado": "sim"
}
```

Este módulo tem como objetivo permitir o acompanhamento diário dos hábitos do usuário, possibilitando análise de progresso e geração de relatórios futuros.

5.4. Módulo de Relatórios

****Responsável:**** Paulo Ricardo da Silva Ferreira
Módulo Responsável pela gestão de relatórios do sistema.

O módulo de relatórios do sistema Habit Flow é responsável por gerar análises e estatísticas sobre usuários, hábitos e registros. Ele permite visualizar informações gerais do sistema, bem como relatórios individuais de desempenho por usuário.

Os dados utilizados são obtidos a partir dos arquivos **usuarios.json**, **habitos.json** e **registros.json**.

Funcionalidades

Relatório Geral (**relatorio_geral**)

Exibe um panorama geral do sistema, incluindo:

- Total de usuários cadastrados;
- Total de hábitos cadastrados;
- Distribuição de hábitos por categoria.

Esse relatório permite uma visão macro do uso do sistema e da organização dos hábitos.

Relatório por Usuário (relatorio_usuario**)**

Apresenta informações detalhadas de um usuário específico, incluindo:

- Dados pessoais (nome, idade, peso e altura);
- Lista de hábitos cadastrados pelo usuário;
- Categoria e meta diária de cada hábito.

O usuário é identificado pelo ID informado no momento da consulta.

Relatório Completo do Usuário (relatorio_completo_usuario**)**

Fornecer uma análise mais aprofundada do desempenho do usuário em seus hábitos, incluindo:

- Lista de hábitos cadastrados;
- Quantidade total de registros por hábito;
- Quantidade de execuções realizadas com sucesso;
- Taxa de conclusão (%) de cada hábito.

Esse relatório permite acompanhar a evolução e disciplina do usuário ao longo do tempo.

Menu de Relatórios (menu_relatorios**)**

Interface de navegação que permite acessar os diferentes tipos de relatórios disponíveis no sistema:

- Relatório geral;
- Relatório por usuário;
- Relatório completo do usuário;
- Retorno ao menu principal.

Estrutura dos Dados Utilizados

Usuários (usuarios.json**)**

```
{
```

```
"id": 1,  
"nome": "João",  
"idade": 20,  
"peso": 70,  
"altura": 1.75  
}
```

Hábitos (**habitos.json**)

```
{  
  "id": 1,  
  "id_usuario": 1,  
  "nome": "Beber água",  
  "categoria": "alimentação",  
  "meta_diaria": "8 copos"  
}
```

Registros (**registros.json**)

```
{  
  "id": 1,  
  "id_usuario": 1,  
  "id_habito": 1,  
  "data": "24/06/2026",  
  "realizado": "sim"  
}
```

Objetivo do Módulo

O módulo de relatórios tem como objetivo transformar os dados registrados no sistema em informações úteis para análise de comportamento, ajudando o usuário a acompanhar sua evolução e identificar padrões em seus hábitos.

6. Persistência de Dados

O sistema Habit Flow utiliza arquivos no formato JSON para persistência de dados, adotando uma abordagem simples em vez de um banco de dados relacional. Cada entidade possui seu próprio arquivo (**usuários.json**, **hábitos.json**, **registros.json**).

O módulo **arquivos.py** centraliza as seguintes funções:

- `carregar_dados(caminho)`: lê o arquivo JSON e retorna uma lista de dicionários. Caso o arquivo não exista, retorna uma lista vazia;
- `salvar_dados(caminho, dados)`: sobrescreve o arquivo com os dados atualizados.

Essa abordagem simplifica o desenvolvimento, eliminando a necessidade de um servidor de banco de dados. No entanto, exige atenção em pontos importantes, como:

- geração correta de IDs (sempre utilizando `max() + 1`, evitando `len()` para não reutilizar IDs);
- garantia da integridade dos dados a cada operação de gravação.

7. Validações e Tratamento de Erros

Todos os módulos seguem um padrão consistente de validação de entrada. São utilizados laços `while True` combinados com blocos `try/except`, evitando que o programa seja interrompido por entradas inválidas, como:

- textos inseridos em campos numéricos;
- campos obrigatórios deixados em branco.

8. Considerações Finais

O desenvolvimento do Habit Flow proporcionou experiência prática relevante na disciplina de Programação I, abordando conceitos como:

- manipulação de arquivos;
- organização modular de código em Python;
- validação de entradas;
- trabalho colaborativo com Git e GitHub, utilizando múltiplas branches e Pull Requests.

Entre os principais pontos positivos, destacam-se:

- a divisão clara de responsabilidades entre os integrantes da equipe, permitindo o desenvolvimento paralelo dos módulos;
- a padronização das funções de validação e persistência, facilitando a integração final do sistema.

Este projeto representa um avanço significativo na compreensão de boas práticas de programação estruturada e modular, servindo como base para o desenvolvimento de sistemas mais complexos no futuro, incluindo a possível migração para bancos de dados relacionais.